Exception Handling in C++ :-

There are 3 types of errors that are occurred in programming:-

- **Syntax Error**
- **Logical Error**
- **Runtime Error**

Syntax Error :- What is a Syntax error? While typing the program, if the programmer mistypes something or did not write something properly then there is an error known as Syntax Error. So, who will identify this error? The compiler will identify this error.

Logical Error :- Suppose you wanted to do something and so wrote the procedure or function or some code but when you run the program the results are different i.e. not as expected.

Then is there any tool for solving this type of problem? Yes, that is a debugger. The debugger will help you to run the program line by line or statement by statement, so in each statement, you can see what is happening and wherever it is going wrong and you can catch it and you can solve that problem.

Note :- The most important thing is that these two types of errors (Syntax Error And Logical Error) are faced by programmers.

Runtime Error :- Suppose you are running some business and you wanted me to develop an application for your business. So, I developed an application and that application is perfect. It is neither having any syntax errors nor having any logical errors. Then I delivered the software to you and you are a user and using that application for your business.

So, the user is going to use the application and while using the application he/she must use it properly. **If he/she mishandled the application then it may cause errors during the runtime of a program.** It's not development time. **At the development time, as a programmer, I have faced logical and syntax errors that I have already removed.** Now it is perfect. There are no errors. And you are using the program. So, at runtime, you are facing errors.

What Could Be the Cause of Error During Runtime?

- First of the reasons for runtime errors is bad input.
- Second, the program that I have given you requires an internet connection but if you are not providing an internet connection then the program cannot continue.
- Third, the program that I have given to you needs some files on your computer system but you have deleted those files or my program is using a printer but you do not have a printer. So, these are the list of problems. **These are nothing but the unavailability of resources**.

All the things that are outside the program or not in the control of the program. Those are in the control of the user.

What happens in the case of runtime errors?

In case of runtime errors, the program crashes or program stops abnormally without completing its execution. Suddenly the program will stop, that's how the runtime errors are dangerous for the user. So, who is responsible? The user is responsible because as a developer I did my job perfectly and the user is not providing proper resources or proper input so the program is failing. Now let us give you some real-life examples.

What can we call these runtime errors?

These runtime errors are called exceptions. Why we are using the term exception? Suppose I have given you a perfect program and it will always run perfectly except in some conditions. Those conditions are giving a bad input or not providing resources. So, the program will not run in those cases. That's why we call these exceptional cases or exceptions.

What are Exceptions?

An exception is nothing but a situation in which we get the runtime error. Let us see in automobile engineering. If there is no fuel in the car then the car will indicate that there is less fuel or it is in a reserved condition so you must find the closest fuel station or gas station.

In the same way. the programs should also behave like that. If there is bad input then the program should not stop suddenly. It should ask the user that **'this input is not proper please enter another type of Input and try again"**. If there is no internet connection then the program should not stop suddenly but it should give a prompt to the user that **"we found that there is no internet connection please connect to the internet and run it again"**. So, **the idea is the program should not terminate under such a situation but guide the user to solve the problem.**

Because who is there to resolve it? The programmer will not come to the client-side and check for the errors and remove them. The program itself can give a proper and meaningful message to the user so that users can remove this type of problem.

Note :- Giving a proper message to the user and informing them about the exact problem. And also providing him guidance to solve that problem, that's it. This is the objective of exception handling.

How to Implement Exception Handling in C++?

```
try {
  // Block of code to try
  throw exception; // Throw an exception when a problem arise
}
catch () {
  // Block of code to handle errors
}
```

Inside the try block, if there is any error then it will jump to the catch block and execute the statements inside the catch block.

- **try**: It represents a block of code that can throw an exception.
- **catch**: It represents a block of code that is executed when a particular exception is thrown.
- **throw**: It is used to throw an exception inside a try block

```cpp
#include<iostream>
using namespace std;
int Division (int a, int b) throw (int){
    if (b == 0)
        throw 1;
    return a / b;
}
int main(){
    int x, y, z;
    cout << "Enter two Numbers:";
    cin >> x >> y;
    try{
        z = Division (x, y);
        cout << z << endl;
    }
    catch (int e){
        cout << "Division by zero " << e << endl;
    }
    cout << "Bye" << endl;
}
//Output
Enter two Numbers:20 0
Division by zero 1
Bye
```

Throwing Exceptions from C++ constructors :-

An exception should be thrown from a C++ constructor whenever an object cannot be properly constructed or initialized. Since there is no way to recover from failed object construction, an exception should be thrown in such cases.

Since C++ constructors do not have a return type, **it is not possible to use return in the codes. Therefore, the best practice is for constructors to throw an exception to signal failure.**

```cpp
#include<iostream>
using namespace std;
class Rectangle{
private:
    int length;
    int breadth;
public:
    Rectangle(int l, int b){
        if (l < 0 || b < 0){
            throw 1;
        }
        else{
            length = l;
            breadth = b;
        }
    }
    void Display()
    {
        cout<<"Length: "<<length<<" Breadth: "<<breadth;
    }
};
int main(){
    try{
        Rectangle r1(10, -5);
        r1.Display();
    }
    catch (int num){
        cout<<"Rectangle Object Creation Failed";
    }
}
//Output
Rectangle Object Creation Failed
```

Key Points regarding exception handling :-

- An exception in C++ is thrown by using the throw keyword from inside the try block. The throw keyword allows the programmer to define custom exceptions.
- Multiple handlers (catch expressions) can be chained – each one with a different exception type. Only the handler whose argument type matches the exception type in the throw statement is executed.
- C++ does not require a finally block to make sure resources are released if an exception occurs.

Using Multiple catch blocks in C++ :-

```cpp
#include<iostream>
#include<conio.h>
using namespace std;
int main(){
    int arr[3] ={-1, 2};
    for (int i = 0; i < 2; i++){
        int num = arr[i];
        try{
            if (num > 0) throw 1;
            else throw 'a';
        }
        catch (int ex){
             cout << "Integer Exception" << endl;
        }
        catch (char ex){
             cout << "Character Exception" << endl;
        }
    }
    return 0;
}
```

Generic Catch Block (Catch - All) in C++ :-

The following example contains a generic catch block to catch any uncaught errors/exceptions. catch(…) block takes care of all types of exceptions.

```cpp
#include <iostream>
#include<conio.h>
using namespace std;
int main(){
    int arr[3] = { -1, 2, 5 };
    for (int i = 0; i < 3; i++){
        int num = arr[i];
        try{
            if(num == -1) throw 1;
            else if(num == 2) throw 'a';
            else throw "Generic";
        }
        catch(int ex){
            cout<<"Integer Exception"<<endl;
        }
        catch (char ex){
            cout<<"Character Exception"<<endl;
        }
        //Generic catch block
        catch (...){ // to catch anytime of exceptions
            cout<<"Generic Exception"<<endl;
        }
    }
    return 0;
}
```

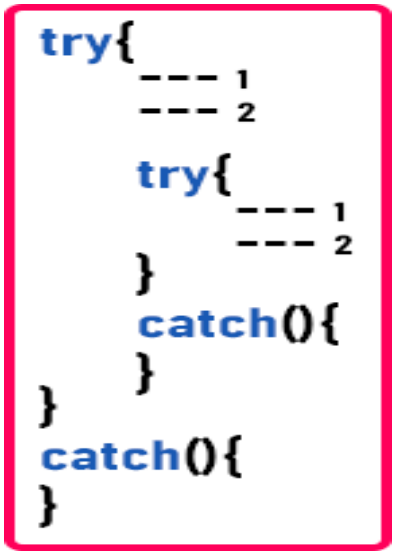
Note : If we write catch-all first, then all the exceptions will be handled here only. The lower catch blocks will never be executed. So, the catch-all block must be the last block.

Throwing User-Defined Type in C++ :-

```cpp
#include<iostream>
#include<conio.h>
using namespace std;
class myExp{
    //Nothing is here
};
int main (){
    int num = 2;
    try{
        if (num == 1) throw 1;
        else if (num == 2) throw myExp();
        else if (num == 3) throw "Unknown Exception";
        else cout<<"Value"<<num<<endl;
    }
    catch(int ex){
        cout<<"Integer Exception"<<endl;
    }
    catch (myExp e){
        cout<<"myExp Exception"<<endl;
    }
    catch (...){
        cout<<"Unknown Exception"<<endl;
    }
    return 0;
}
int main (){
    int num = 2;
    try{
        if (num == 1) throw 1;
        else if (num == 2) throw myExp();
        else if (num == 3) throw "Unknown Exception";
        else cout<<"Value"<<num<<endl;
    }
    catch(int ex){
        cout<<"Integer Exception"<<endl;
    }
    catch(myExp e){
        cout<<"myExp Exception"<<endl;
    }
    catch(...){
        cout<<"Unknown Exception"<<endl;
    }
    return 0;
}
```

Can we have a Try Block inside a Try block?

Yes, we can have a try block inside another try block in C++. The following diagram shows the syntax of writing nested try blocks in C++.



Example to Understand Try-Catch Blocks in C++:

```
class MyExp1{};

class MyExp2 : public MyExp1{};
```

So, we have these two classes. And MyExp2 is publicly inherited from the MyExp1 class. So, MyExp1 is a parent class and MyExp2 is a child class.

```
try{
    ---
    ---
    ---
}
catch(MyExp1 e){
}
catch(MyExp2 e){
}
```

```
try{
    ---
    ---
    ---
}
catch(MyExp2 e){
}
catch(MyExp1 e){
}
```

As you can see, we have written catch blocks for both types of exceptions. So, is this the correct format of catch block? No. We have written the catch block for the parent class first and then for the child class. **We must write catch blocks for the child class first and then for the parent class as shown in the below image.**

Inheriting Exception Class from Built-in Exception Class :-

If you are throwing your own class exception then better extend your class from the built-in exception class in C++ as follows. **But it is not mandatory.**

```
class MyException: public exception {
};
```

Overriding the exception class what method in C++ :-

After inheriting our custom exception class from the built-in exception class, do we have to override anything? Yes, we can override one method which is **"what"** method as follows. But this is not mandatory as in our previous example we have not overridden the what method and it is working fine.

```
class MyException : public exception{
        public:
                char * what(){
                        return "My Custom Exception";
                }
};
```

How to make the function throws something in C++?

```
int Division(int a, int b) throw (MyException)
{
      if (b == 0)
          throw MyException();
      return a / b;
}
```

```
int Division(int a, int b) throw (int)
{
        if (b == 0)
            throw 1;
        return a / b;
}
```

So, whatever the type of value you are throwing, you can mention that in the brackets. **And if there are more values then you can mention them with commas as follows:**

```
int Division(int a, int b) throw (int, MyException)
{
        if (b == 0)
            throw 1;
        return a / b;
}
```

```cpp
#include<iostream>
using namespace std;
#include <exception>
class MyException:public exception{
public:
    char * what(){
        return "My Custom Exception";
    }
};
int Division(int a, int b) throw (int, MyException){
    if (b == 0) throw 1;
    if (b == 1) throw MyException();
    return a / b;
}
int main(){
    int x = 10, y = 1, z;
    try{
        z = Division(x, y);
        cout<<z<<endl;
    }
    catch(int x){
        cout<<"Division By Zero Error"<<endl;
    }
    catch (MyException ME){
        cout<<"Division By One Error"<<endl;
        cout<<ME.what ()<<endl;
    }
    cout<<"End of the Program"<<endl;
}
```

Can we use cout instead of throw?

If you use cout, the user will know the error. **And if you use throw, it will inform the calling function about the error.**

Are return and throw the same?

Return is for returning results. The throw is for reporting an error. If you change their roles then the roles of Try and Catch will also change.

Template in C++

Template in C++ allows us to define Generic Functions and Generic Classes. That means using a Template we can implement Generic Programming in C++. They are going to work with a variety of data types. Templates in C++ can be represented in two ways. They are as follows.

- **Function templates**
- **Class templates**

```
int max(int a, int b){
        return a > b ? a : b;
}
```

```
T max(T a, T b){
        return a > b ? a : b;
}
```

```
#include<iostream>
using namespace std;
template <class T, class R>
void Add (T x, R y){
    cout << x + y << endl;
}
int main(){
    //Integer and Integer
    Add (4, 24);
    //Float and Float
    Add (25.7f, 67.6f);
    //Integer and double
    Add (14, 25.5);
    //Float and Integer
    Add (25.7f, 45);
    return 0;
}
//Output
28
93.3
39.5
70.7
```

```cpp
#include <iostream>
using namespace std;
template<class T>
class Stack{
    private:
        T * stk;
        int top, size;
    public:
        Stack (int sz){
            size = sz;
            top = -1;
            stk = new T[size];
        }
        void Push(T x);
        T Pop();
};
template<class T>
void Stack<T>::Push(T x){
    if (top == size - 1)
        cout<<"Stack is Full"<<endl;
    else{
        top++;
        stk[top] = x;
        cout<<x <<" Added to Stack"<<endl;
    }
}
template<class T>
T Stack<T>::Pop(){
    T x = 0;
    if (top == -1)
        cout<<"Stack is Empty"<<endl;
    else{
        x = stk[top];
        top--;
        cout<<x<<" Removed from Stack"<<endl;
    }
    return x;
}
int main(){
    //Stack of Integer
    Stack<float> stack1(10);
    stack1.Push(10), stack1.Push(23),stack1.Push(33);
    stack1.Pop();
    //Stack of double
    Stack<double> stack2(10);
    stack2.Push(10.5), stack2.Push(23.7), stack2.Push(33.8);
    stack2.Pop();
    return 0;
}
```